# CODE COVERAGE REPORT

## Electronic Store Inventory Management System

**Academic Software Engineering Testing Deliverable** | Executable Statement Coverage Assessment

## 1. Introduction

Code coverage is an objective quantitative metric in software engineering that measures the degree to which the executable source code of an application is invoked when an automated test suite is run. Statement coverage (or line coverage) specifically tracks the execution of individual source instructions, identifying which lines of code were evaluated during test execution and which lines remained unreached.

This report documents the statement coverage achieved by executing the complete 373-test automated test suite across the production source code of the Electronic Store Inventory Management System. The measurement assesses all business logic, presentation routing, data access models, and operational utility scripts. Statement coverage analysis provides transparency into untested failure paths, redundant error handling, and critical execution paths requiring regression protection.

## 2. Objectives of Code Coverage Measurement

- **Measure Executable Statement Coverage:** Determine the exact proportion of application statements exercised during automated test execution across all functional layers.
- **Identify Uncovered Source Statements:** Pinpoint the precise file locations and line numbers that were bypassed during test execution to reveal untested application logic.
- **Evaluate Module-Wise Testing Rigor:** Assess the depth of testing across Presentation, Business Logic, and Data Access layers to uncover disparities in test thoroughness.
- **Strengthen Regression Resistance:** Ensure that critical business transactions—such as inventory decrements, financial calculations, purchase order workflows, and security authentications—are thoroughly exercised.
- **Guide Test Suite Refinement:** Provide actionable empirical data to guide the creation of targeted tests for unreached exception handlers and defensive validation guards.

## 3. Testing and Coverage Environment

Coverage measurement was performed directly within the local repository virtual environment using industry-standard Python testing and coverage instrumentation tools. Fresh execution ensured that coverage statistics were compiled without interference from stale coverage caches or previous run artifacts.

| Component / Tool | Specification / Version |
|---|---|
| **Programming Language** | Python 3.14.6 (64-bit) |
| **Web Application Framework** | Flask 3.1.0 |
| **Test Execution Engine** | Pytest 9.1.1 |
| **Coverage Instrumentation Tool** | coverage.py 7.17.2 via pytest-cov 7.1.0 |
| **ORM & Database Engine** | SQLAlchemy 2.0 / SQLite 3 (isolated ephemeral test fixtures) |
| **Operating System** | macOS and Windows (both were used) |
| **Virtual Environment Path** | ./venv (Isolated Project Virtualenv) |
| **Measurement Invocation** | pytest --cov=app --cov=scripts --cov-report=term-missing |

## 4. Coverage Measurement Methodology and Scope

The coverage instrumentation was configured to measure strictly the production source code belonging to the system. The measurement boundary encompasses exactly two source directories: app/ (containing the core application architecture) and scripts/ (containing operational automation utilities). All third-party libraries within venv/, external packages, temporary build directories, and the test suite itself (tests/) were strictly excluded from measurement.

The measured scope contains exactly **28 Python source files**. Each statement was dynamically monitored during the execution of all 373 automated tests. Execution counts, covered statements, and missing statement line numbers were captured into a machine-readable JSON coverage report and reconciled against terminal output.

## 5. Overall Code Coverage Results

The empirical statement coverage results compiled from the fresh execution of the complete automated test suite are detailed below:

| Coverage Metric | Empirical Value | Engineering Interpretation |
|---|---|---|
| **Total Executable Statements** | 1,996 | Total executable statements across 28 source files |
| **Covered Statements** | 1,950 | Statements executed at least once during test suite run |
| **Missing Statements** | 46 | Statements unreached during test suite execution |
| **Overall Statement Coverage (Exact)** | **97.70%** | 1,950 / 1,996 = 97.69539% |
| **Overall Statement Coverage (Display)** | **98%** | Rounded display integer per coverage.py standard |
| **Automated Tests Executed** | 373 passed / 0 failed | 100% test pass rate across 20 test modules |

## 6. Module-Wise Code Coverage Breakdown

The following table provides an exhaustive account of statement coverage across all 28 measured source modules. Modules are grouped logically by architectural tier. Uncovered line numbers are recorded exactly as reported by coverage.py.

| Source File Path | Total Stmts | Covered | Miss | Line Cov % | Uncovered Line Numbers |
|---|---|---|---|---|---|
| **app/__init__.py** | 62 | 62 | 0 | 100.00% | None (100% covered) |
| **app/config.py** | 13 | 13 | 0 | 100.00% | None (100% covered) |
| **app/business/__init__.py** | 4 | 4 | 0 | 100.00% | None (100% covered) |
| **app/business/audit.py** | 8 | 8 | 0 | 100.00% | None (100% covered) |
| **app/business/password_reset_email.py** | 20 | 20 | 0 | 100.00% | None (100% covered) |
| **app/business/purchase_order.py** | 267 | 250 | 17 | 93.63% | 28, 43, 146, 194, 206, 211, 220, 237, 270, 282, 287, 305, 337, 359, 373, 387, 439 |
| **app/business/report_exports.py** | 46 | 46 | 0 | 100.00% | None (100% covered) |
| **app/business/sales.py** | 75 | 73 | 2 | 97.33% | 70, 124 |
| **app/business/security.py** | 21 | 21 | 0 | 100.00% | None (100% covered) |
| **app/business/stock.py** | 128 | 127 | 1 | 99.22% | 21 |
| **app/data/__init__.py** | 0 | 0 | 0 | N/A | N/A (0 statements) |
| **app/data/extensions.py** | 4 | 4 | 0 | 100.00% | None (100% covered) |
| **app/data/models/__init__.py** | 8 | 8 | 0 | 100.00% | None (100% covered) |
| **app/data/models/audit_log.py** | 12 | 12 | 0 | 100.00% | None (100% covered) |
| **app/data/models/product.py** | 32 | 31 | 1 | 96.88% | 36 |
| **app/data/models/purchase_order.py** | 72 | 72 | 0 | 100.00% | None (100% covered) |
| **app/data/models/sale.py** | 20 | 20 | 0 | 100.00% | None (100% covered) |
| **app/data/models/stock.py** | 22 | 22 | 0 | 100.00% | None (100% covered) |
| **app/data/models/supplier.py** | 23 | 23 | 0 | 100.00% | None (100% covered) |

| Source File Path | Total Stmts | Covered | Miss | Line Cov % | Uncovered Line Numbers |
|---|---|---|---|---|---|
| **app/data/models/user.py** | 16 | 16 | 0 | 100.00% | None (100% covered) |
| **app/presentation/__init__.py** | 0 | 0 | 0 | N/A | N/A (0 statements) |
| **app/presentation/authz.py** | 19 | 19 | 0 | 100.00% | None (100% covered) |
| **app/presentation/routes/__init__.py** | 0 | 0 | 0 | N/A | N/A (0 statements) |
| **app/presentation/routes/auth.py** | 182 | 180 | 2 | 98.90% | 146-147 |
| **app/presentation/routes/main.py** | 568 | 561 | 7 | 98.77% | 60, 79, 86, 146, 311-313 |
| **app/presentation/routes/purchase_order.py** | 213 | 197 | 16 | 92.49% | 59, 61, 94-95, 226-227, 267, 269, 278, 301-302, 337-338, 382, 439-440 |
| **app/presentation/routes/supplier.py** | 110 | 110 | 0 | 100.00% | None (100% covered) |
| **scripts/backup_database.py** | 51 | 51 | 0 | 100.00% | None (100% covered) |
| **Total / Aggregate** | **1,996** | **1,950** | **46** | **97.70%** | **46 Missing Lines Across 7 Modules (21 Modules at 100%)** |

## 7. Coverage Analysis and Uncovered Code Assessment

With an aggregate statement coverage of **97.70%**, 21 of the 28 measured modules achieve 100% executable statement coverage (with 3 package initializers correctly marked as N/A due to containing zero executable statements). The remaining 46 uncovered statements reside within 7 specific modules. A detailed engineering review reveals the rationale for why these specific statements remain unreached:

- **Purchase Order Service (app/business/purchase_order.py — 17 missing statements, 93.63%):** The unreached lines represent defensive exception recovery branches and secondary rollback clauses. Specifically, lines 28, 43, 146, 194, 206, 211, 220, 237, 270, 282, 287, 305, 337, 359, 373, 387, and 439 guard against catastrophic database write failures during status transitions (e.g., catching unexpected SQLAlchemy operational errors during order receipt or line item deletion after primary validations have passed).
- **Purchase Order Presentation Routes (app/presentation/routes/purchase_order.py — 16 missing statements, 92.49%):** Unreached lines (59, 61, 94-95, 226-227, 267, 269, 278, 301-302, 337-338, 382, 439-440) comprise defensive HTTP error redirects, nested form parsing fallbacks when optional query strings are completely omitted, and secondary flash warning handlers triggered only when dynamic client JavaScript submits malformed multipart payloads.
- **Main Presentation Routes (app/presentation/routes/main.py — 7 missing statements, 98.77%):** Lines 60, 79, 86, 146, and 311-313 represent defensive category lookup fallbacks and error redirection logic when an invalid query string parameter is supplied to product search pagination.
- **Sales Service & Presentation Routes (app/business/sales.py & app/presentation/routes/auth.py):** In sales (lines 70, 124), unreached lines handle fallback cash change calculations when customer payment is zero. In auth (lines 146-147), unreached statements correspond to invalid token payload exceptions during password reset.
- **Product Model & Stock Service (app/data/models/product.py & app/business/stock.py):** Line 36 in product model handles edge-case price string formatting when price is null. Line 21 in stock service handles fallback return when minimum reorder threshold is negative.

## 8. Conclusion and Coverage Limitations

The empirical measurement confirms that the automated test suite achieves **97.70% statement coverage** across the production codebase, substantially exceeding standard software engineering industry targets (typically 80%–90%). All critical architectural pathways—including POS checkout, product catalog management, supplier administration, purchase order processing, and database backup routines—are rigorously exercised by automated tests.

**Important Theoretical Limitations of Statement Coverage:** High statement coverage is an essential quality indicator, but it does not mathematically prove software correctness:

- **Absence of Semantic Verification:** Executing a statement confirms that the instruction executed without throwing an unhandled exception; it does not independently prove that the business calculation satisfies all domain requirements.
- **Combinatorial Decision Paths:** Statement coverage does not evaluate combinations of boolean conditions or alternate decision branches. (Decision-branch evaluation is addressed separately in the dedicated Branch Coverage Report).
- **Concurrency and Race Conditions:** High statement coverage does not detect thread contention, SQLite lock timeouts, or race conditions that may arise under multi-user production environments.